

Machine Learning

Lecture 13: Embeddings

Iurii Efimov



Outline

1. NLP introduction
2. Word embeddings
3. Word2vec
4. Image embeddings

Text features

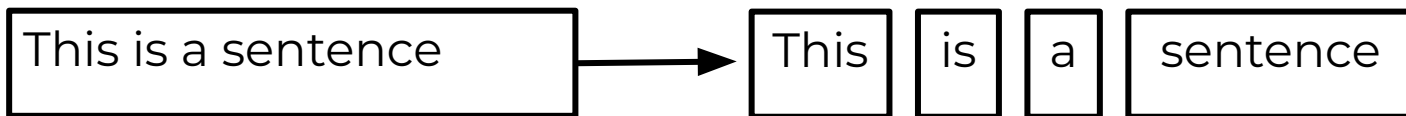
girafe
ai

01



Tokenization

- Split the input into tokens
- Tokens:
 - words
 - symbol
 - morpheme
 - ...





Bag-of-words

the dog is on the table

0	0	1	1	0	1	1	1
are	cat	dog	is	now	on	table	the

Problems

- No information about words order
- Word vectors are huge and sparse
- Word vectors are not normalized
- Same words can take different forms
 -



TF-IDF

- **Term Frequency (tf):** gives us the frequency of the word in each document in the corpus.

$$\text{tf}(t, d) = f_{t, d}$$

- **Inverse Document Frequency (idf):** used to calculate the weight of rare words across all documents in the corpus. The words that occur rarely in the corpus have a high IDF score.

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

N : total number of documents in the corpus $N = |D|$

$|\{d \in D : t \in d\}|$: number of documents where the term t

TF-IDF: Example



- **Sentence A:** The car is driven on the road.
- **Sentence B:** The truck is driven on the highway

(each sentence is a separate document)

TF-IDF: Example



Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7			
Car	1/7	0			
Truck	0	1/7			
Is	1/7	1/7			
Driven	1/7	1/7			
On	1/7	1/7			
The	1/7	1/7			
Road	1/7	0			
Highway	0	1/7			

TF-IDF: Example



Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2)=0$		
Car	1/7	0	$\log(2/1)=0.3$		
Truck	0	1/7	$\log(2/1)=0.3$		
Is	1/7	1/7	$\log(2/2)=0$		
Driven	1/7	1/7	$\log(2/2)=0$		
On	1/7	1/7	$\log(2/2)=0$		
The	1/7	1/7	$\log(2/2)=0$		
Road	1/7	0	$\log(2/1)=0.3$		
Highway	0	1/7	$\log(2/1)=0.3$		

TF-IDF: Example



Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2)=0$	0	0
Car	1/7	0	$\log(2/1)=0.3$	0.043	0
Truck	0	1/7	$\log(2/1)=0.3$	0	0.043
Is	1/7	1/7	$\log(2/2)=0$	0	0
Driven	1/7	1/7	$\log(2/2)=0$	0	0
On	1/7	1/7	$\log(2/2)=0$	0	0
The	1/7	1/7	$\log(2/2)=0$	0	0
Road	1/7	0	$\log(2/1)=0.3$	0.043	0
Highway	0	1/7	$\log(2/1)=0.3$	0	0.043

TF-IDF: Example



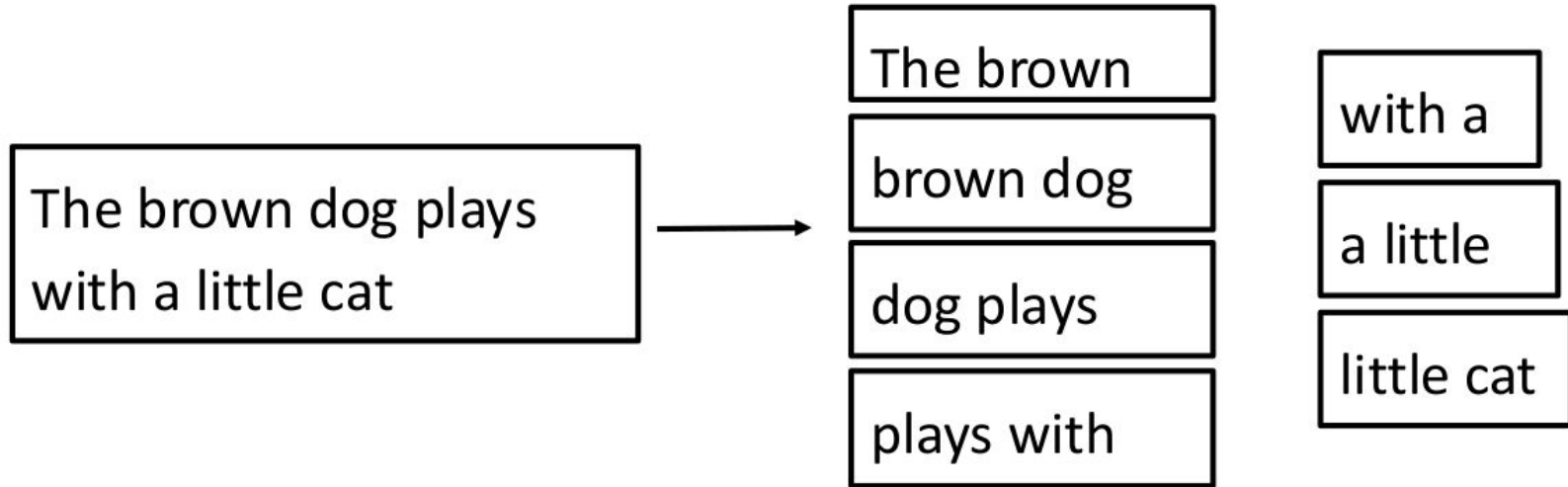
```
from sklearn.feature_extraction.text import TfidfVectorizer
```



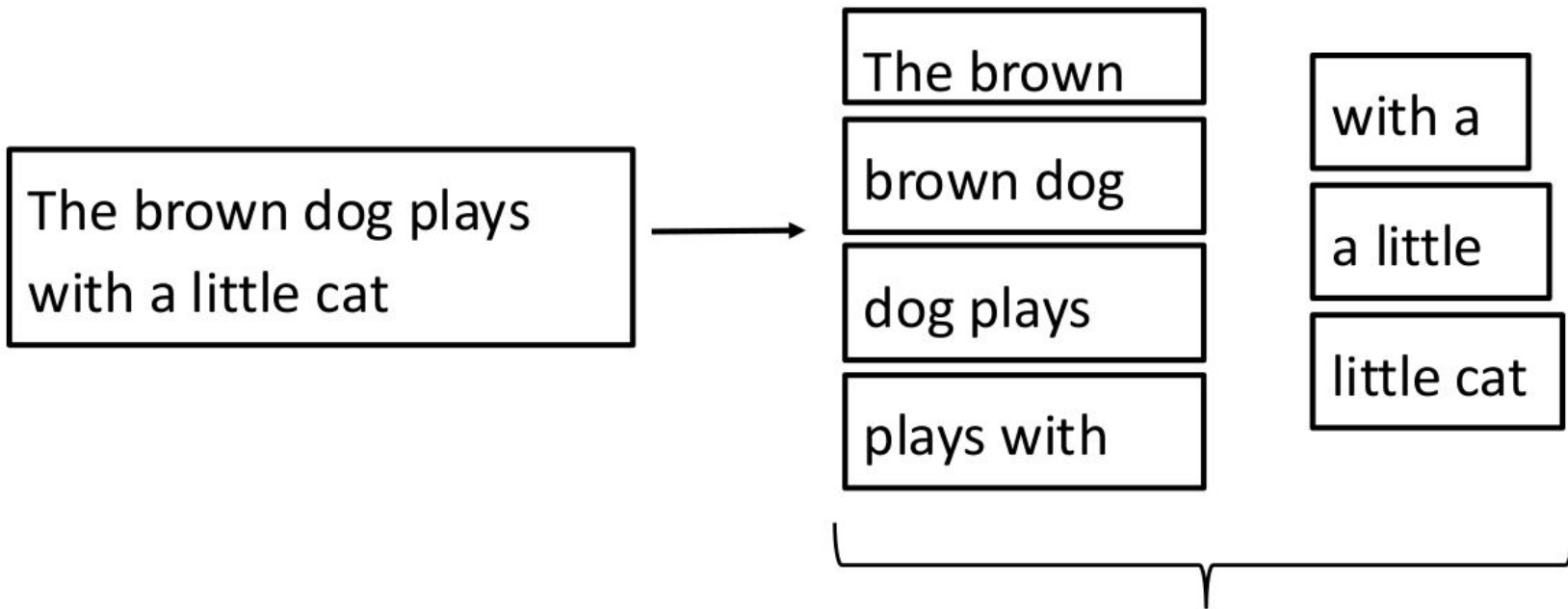
Bag-of-words

How to improve BOW?

- Use n-gramms instead of words!

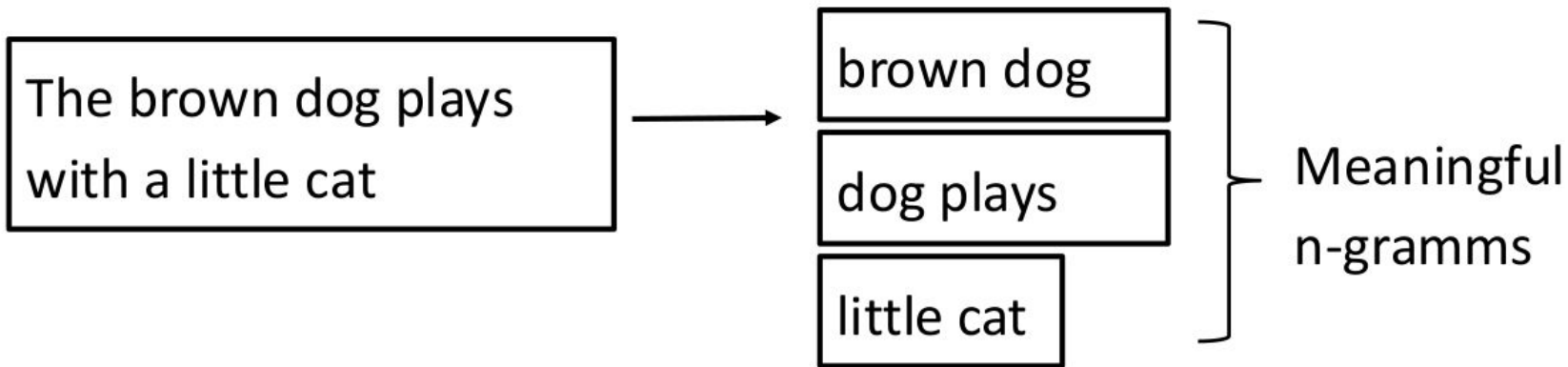


Bag-of-words



Do we need all this bigramms?

Bag-of-words



Meaningful n-gramms are often called **collocations**

Word Embeddings

girafe
ai

02

One-hot vectors



Problems:

- Huge vectors
- VERY sparse
- No semantics or word similarity information included

Rome Paris word V

Rome = [1, 0, 0, 0, 0, 0, ..., 0]

Paris = [0, 1, 0, 0, 0, 0, ..., 0]

Italy = [0, 0, 1, 0, 0, 0, ..., 0]

France = [0, 0, 0, 1, 0, 0, ..., 0]

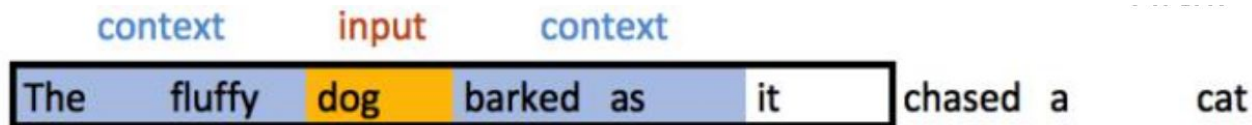


Distributional semantics

Does vector similarity imply semantic similarity?

“You shall know a word by the company it keeps”

Firth

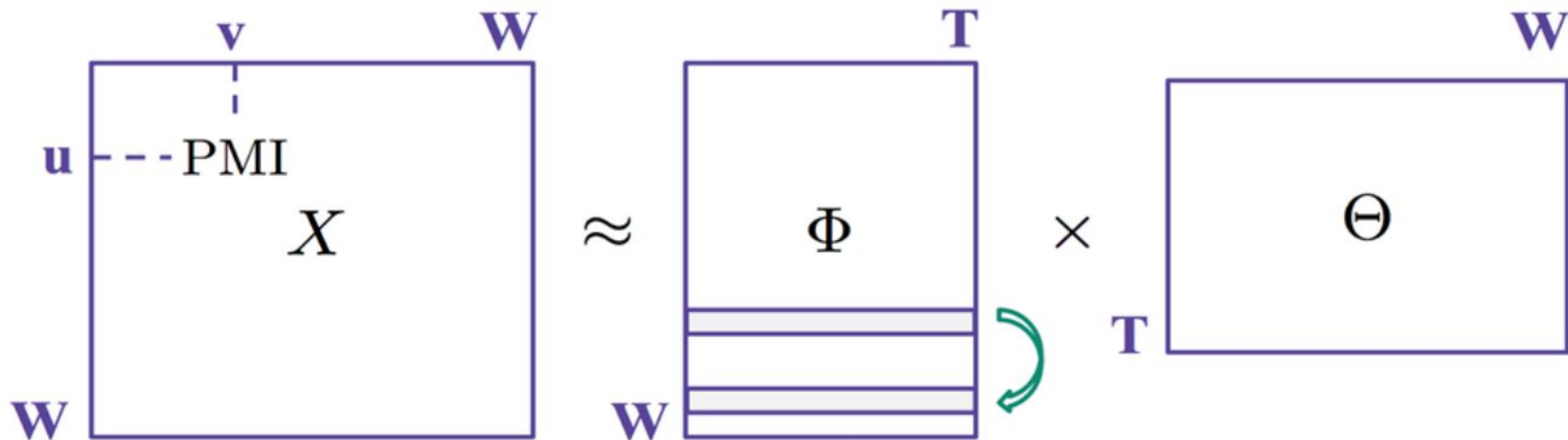


Matrix factorization



- Input: PMI, word cooccurrences, etc.
- Method: dimensionality reduction (SVD)
- Output: word similarities

$$PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$





Collocations: first step

- **Delete:**

- High-frequency n-gramms
 - Articles, prepositions
 - Auxiliary verbs (to be, to have, etc.)
 - General vocabulary
- Low-frequency n-gramms
 - Typos
 - Combinations that occur 1-2 times in a text

Collocations: context is all you need



- Co-Occurrence counters in a window of fixed size
 - states for the number of times we've seen word u and word v together in the window
- Better solution: Pointwise Mutual Information (PMI)

$$PMI = \log \frac{p(u, v)}{p(u)p(v)} = \log \frac{n_{uv}n}{n_u n_v}$$

- Much better solution: Positive PMI (pPMI)

$$pPMI = \max(0, PMI)$$

Learning word vectors

girafe
ai

03

Embeddings: intuition



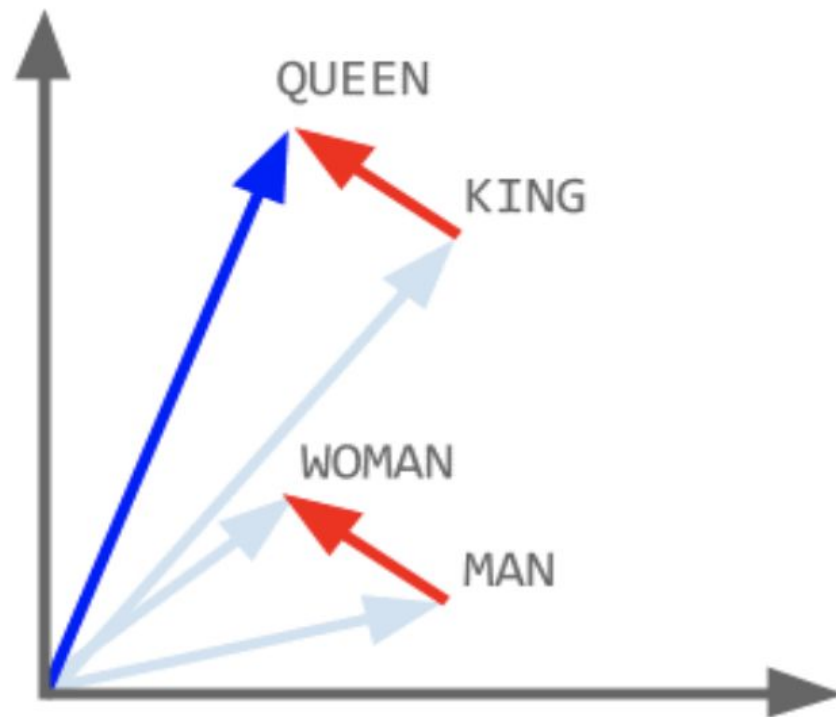
What is king - man + woman?



Embeddings: intuition



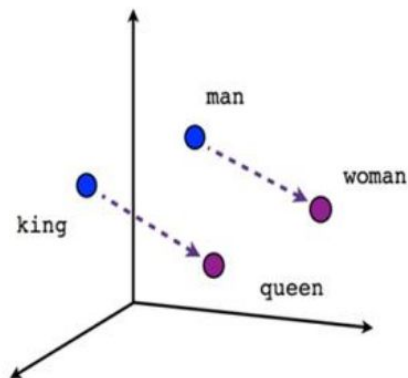
So $\text{king} - \text{man} + \text{woman} = \text{queen!}$



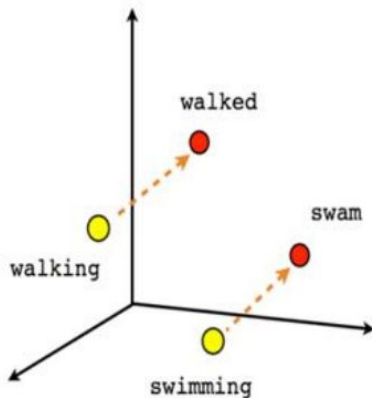
Word2vec



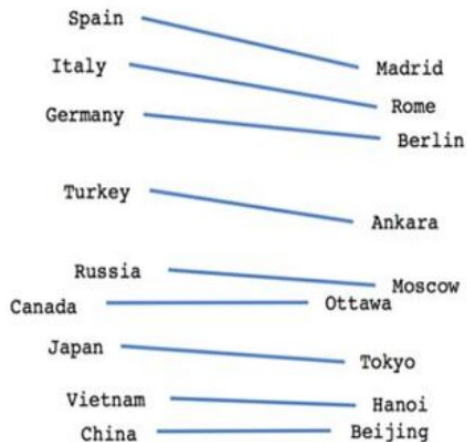
Word2vec (Mikolov et al. 2013) - a framework for learning word embeddings



Male-Female



Verb tense



Country-Capital

Word2vec

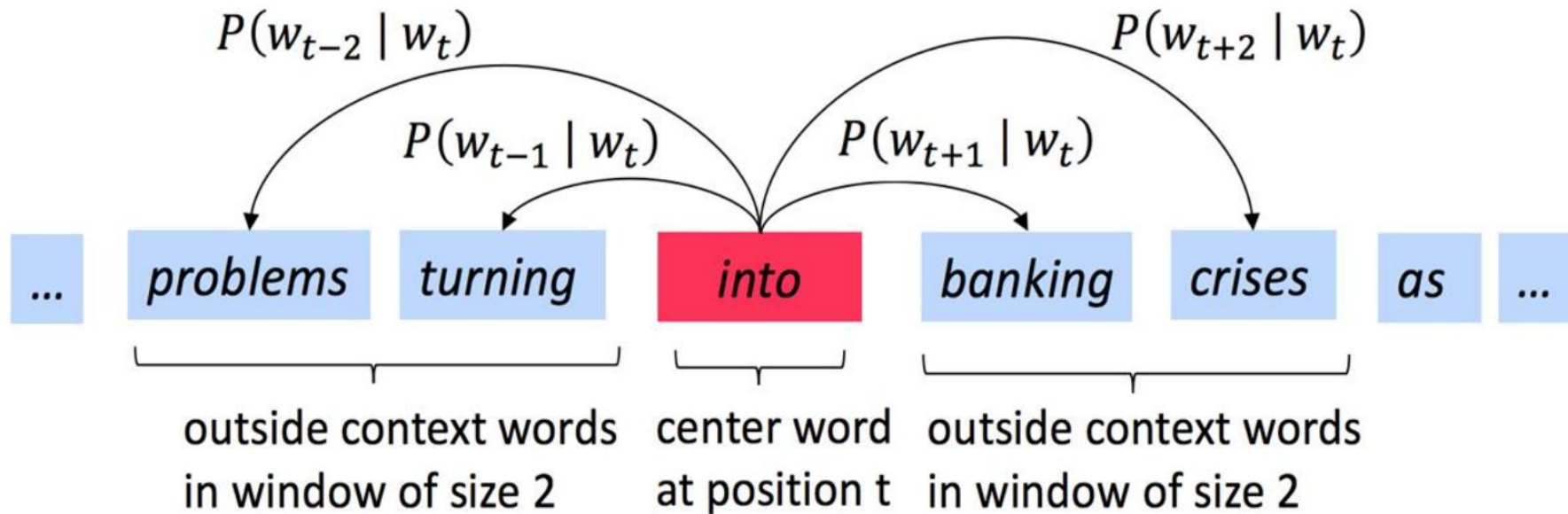


Source Text

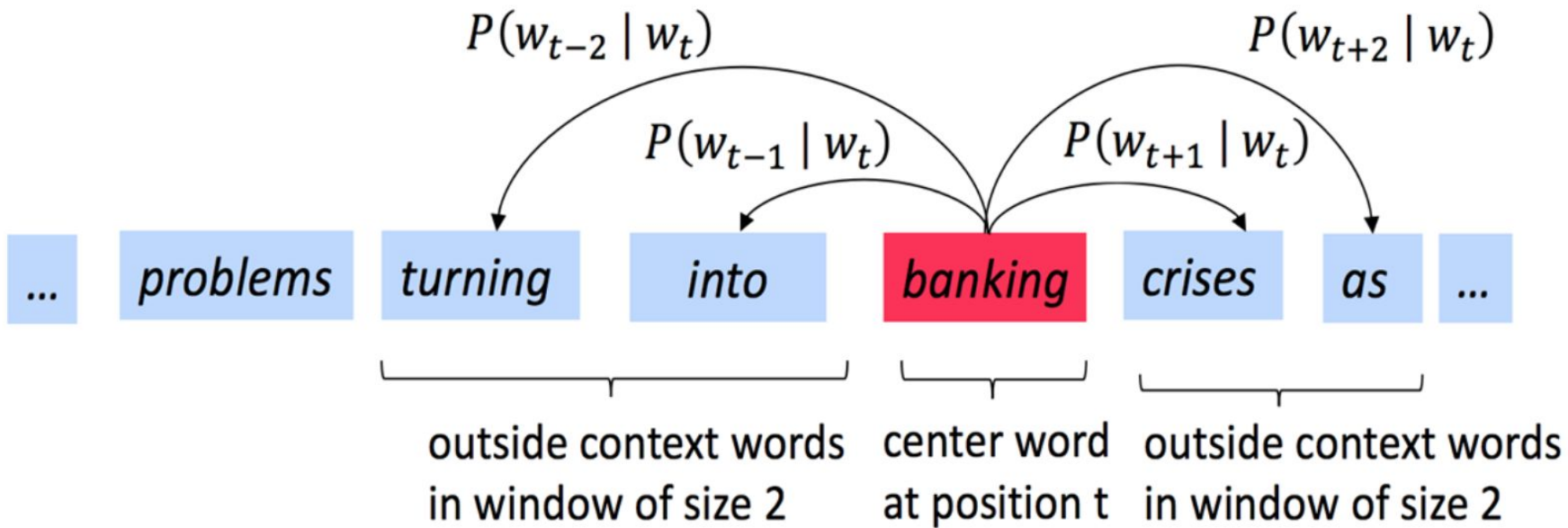
Training Samples

The quick brown fox jumps over the lazy dog. →	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog. →	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog. →	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog. →	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

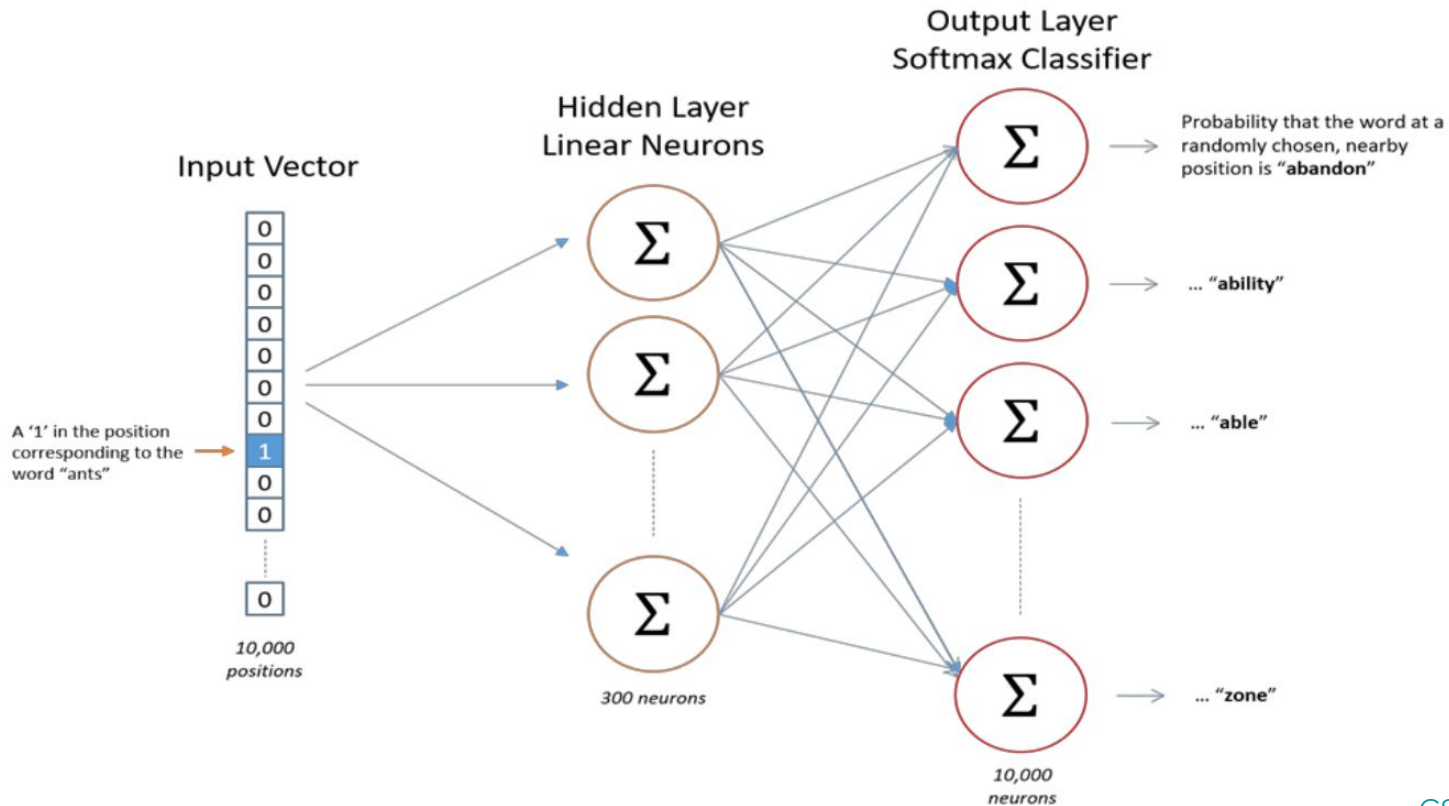
Word2vec



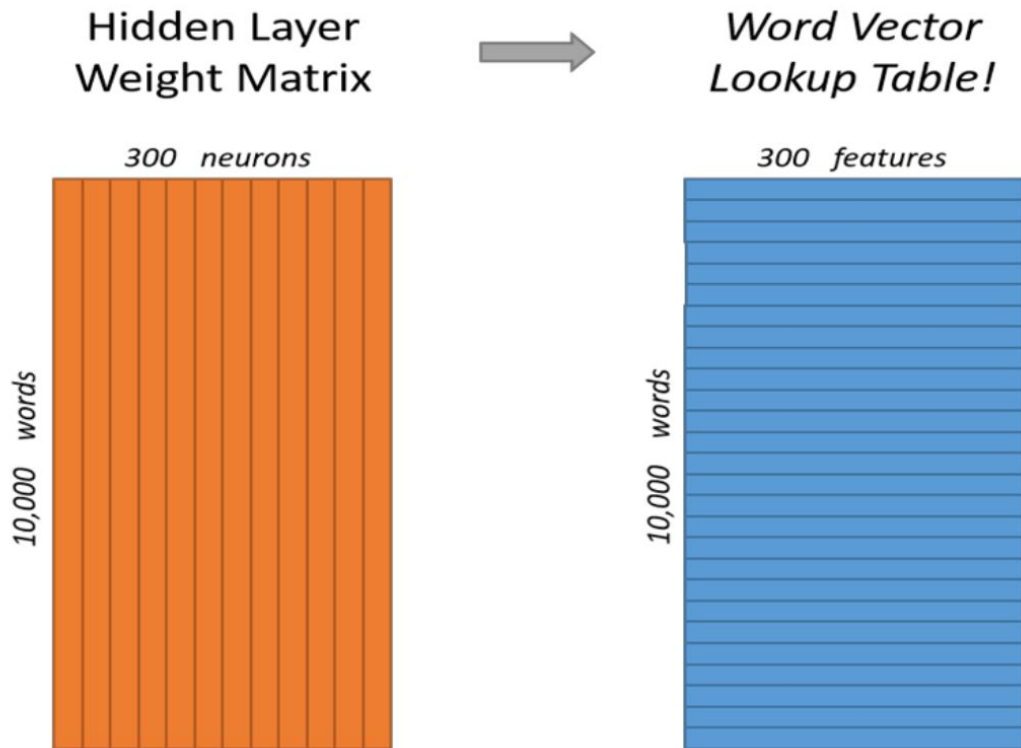
Word2vec



Word2vec



Word2vec





Word2vec

- Word vectors with 300 components
- Vocabulary of 10,000 words.
- Weight matrix with $300 \times 10,000 = 3$ million weights each!
- Training is too long and computationally expensive

HOW TO FIX IT?



Word2vec

Basic approaches:

1. Treating common word pairs or phrases as single “words” in their model.
2. Subsampling frequent words to decrease the number of training examples.
3. Modifying the optimization objective with a technique they called “Negative Sampling”, which causes each training sample to update only a small percentage of the model’s weights.

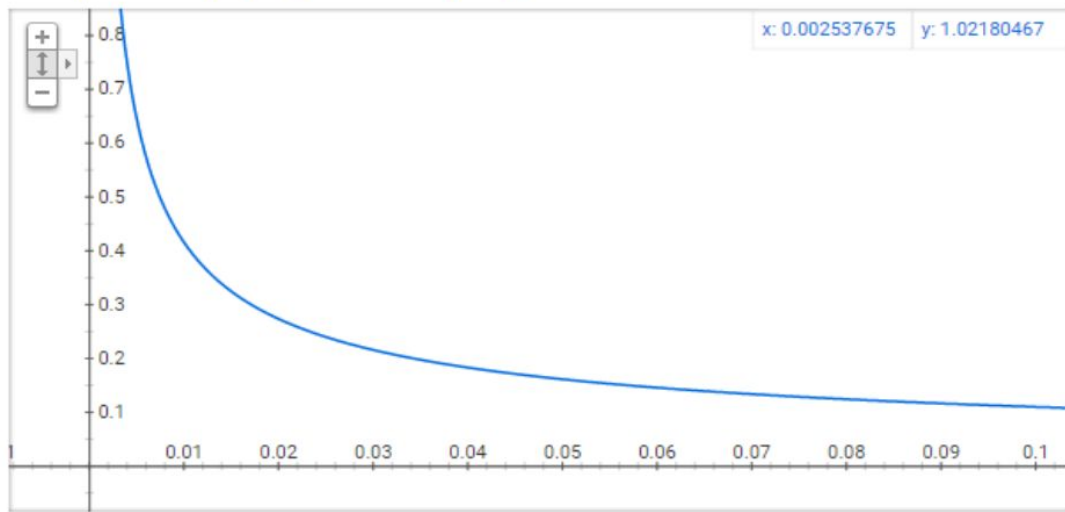


Word2vec: Subsampling

Subsampling frequent words.

w_i is the word, $z(w_i)$ is the fraction of this word in the text

Graph for $(\sqrt{x/0.001}+1)*0.001/x$



$P(w_i)$ is the probability of
keeping the word:

$$P(w_i) = \left(\sqrt{\frac{z(w_i)}{0.001}} + 1 \right) \cdot \frac{0.001}{z(w_i)}$$



Word2vec: Negative Sampling

- Error is computed only for a few rows. All other words have zero error, so no updates by the backprop mechanism.
- More frequent words are selected to be negative samples more often. The probability for selecting a word is just its weight divided by the sum of weights for all words.

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=0}^n (f(w_j)^{3/4})}$$



Word2vec: two models

Continuous BOW (CBOW)

Predict center word from (bag of) context words

$$p(w_i | w_{i-h}, \dots, w_{i+h})$$

- Predict one word each time
- Relatively fast

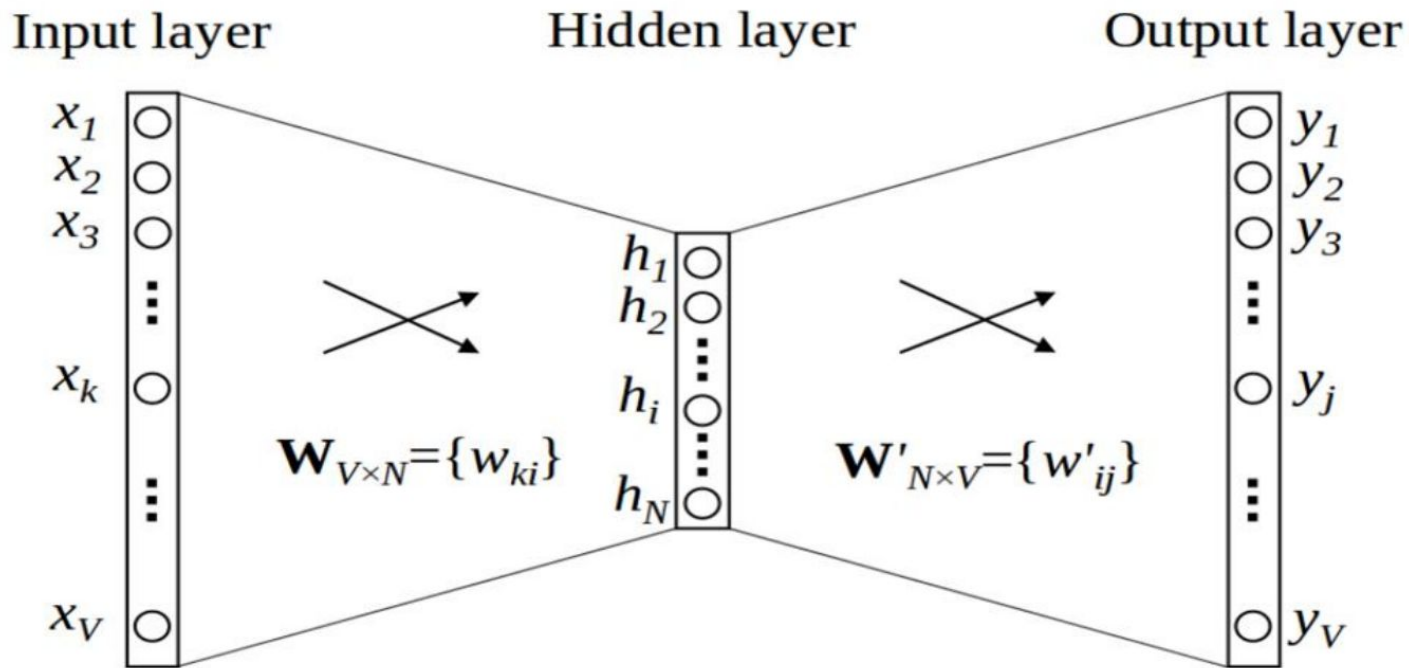
Skip-gram

Predict context ("outside") words (position independent) given center word

$$p(w_{i-h}, \dots, w_{i+h} | w_i)$$

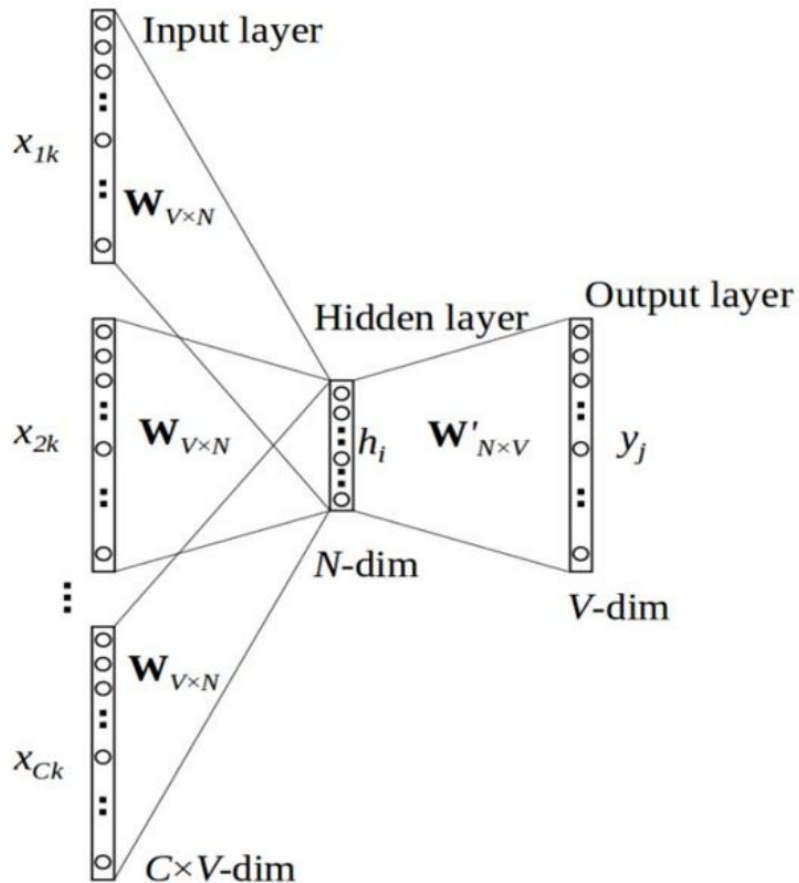
- Predicting context by one word
- Much slower
- Better with infrequent words

Word2vec: Skip-gram





Word2vec: CBOW





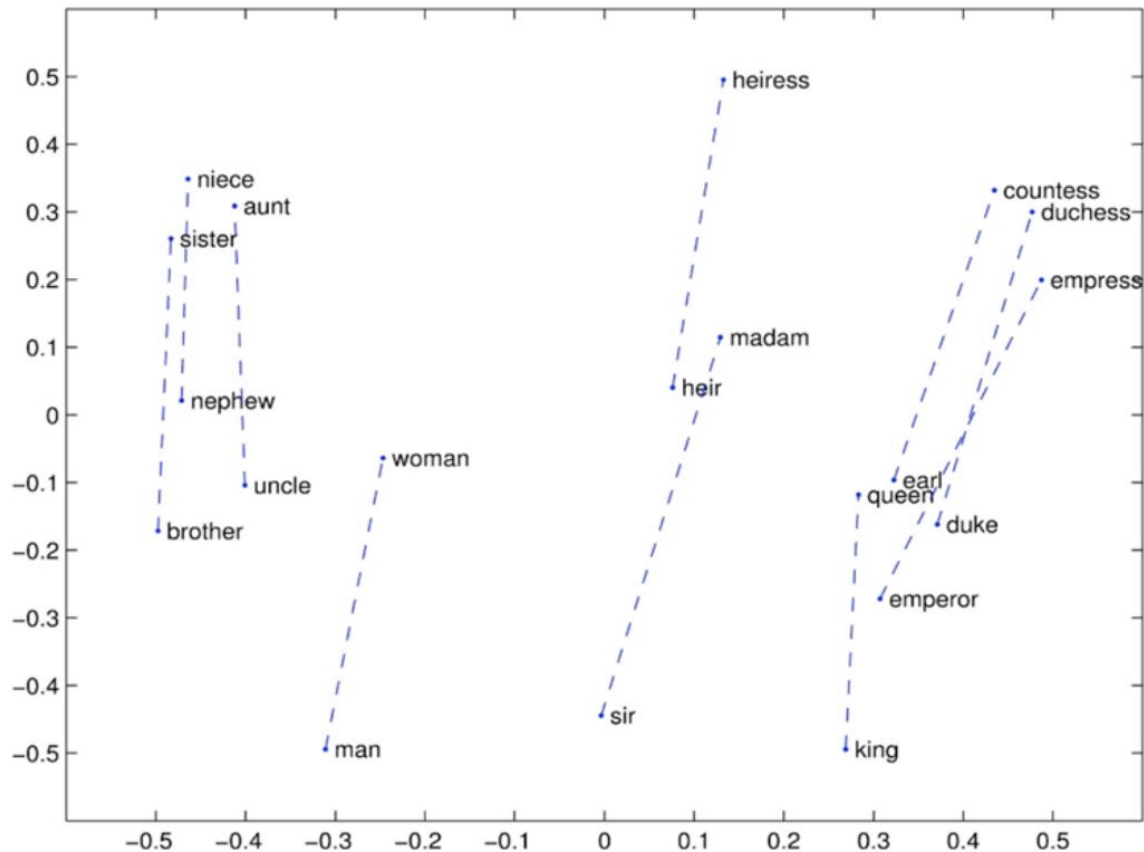
Word2vec: Contrastive

Simplify the approaches above:

Consider only one **positive pair** of words and **a few negative pairs**.

$$\log \sigma(v_{w_o}^\top v_{w_t}) + \sum_{\substack{i=0 \\ w_i \in P_n(\omega)}}^k \log \sigma(-v_{w_i}^\top v_{w_t})$$

GloVe: visualizations

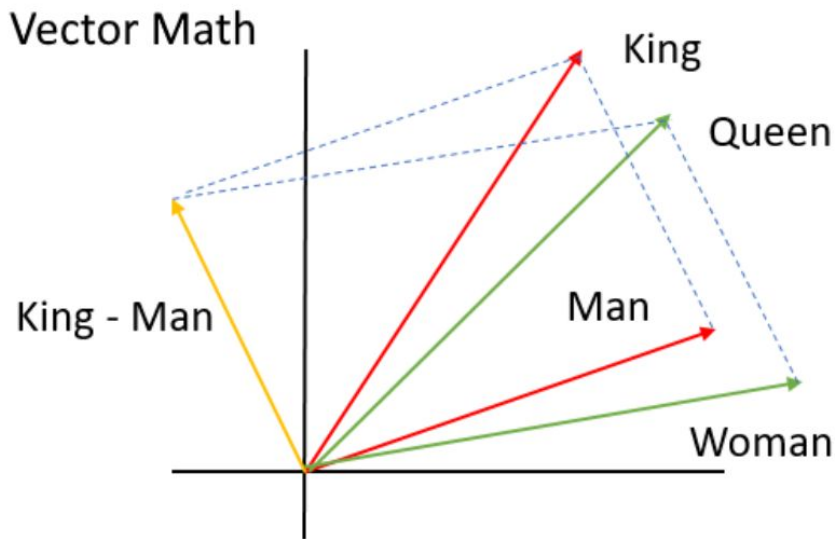


Word2vec: word analogies



King - man + woman = queen
↓ ↓ ↓ ↓
 x y y' $target$

$\cos(x - y + y', target) \rightarrow \max_{target}$



Learning image vectors

girafe
ai

04

Computer vision tasks



Classification



CAT

No spatial context

Semantic segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Computer vision tasks



Object Detection

Instance segmentation



DOG, DOG, CAT



DOG, DOG, CAT

Multiple objects

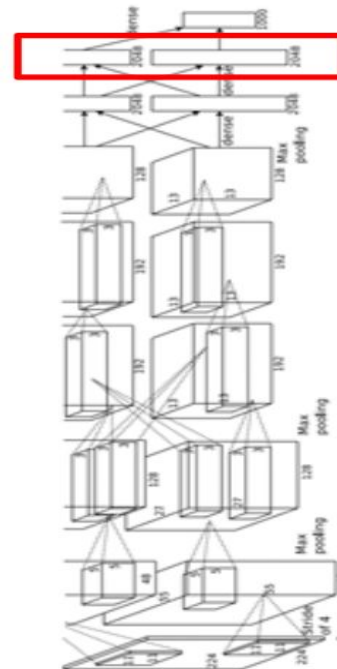
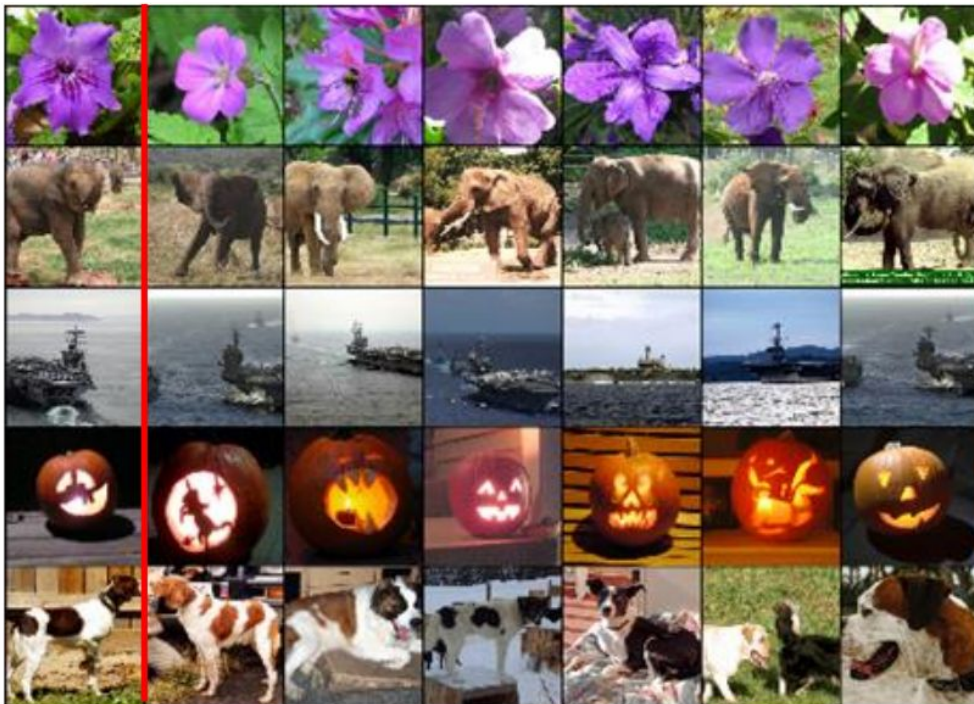
Trained model embeddings



Test image

L2 nearest neighbors in feature space

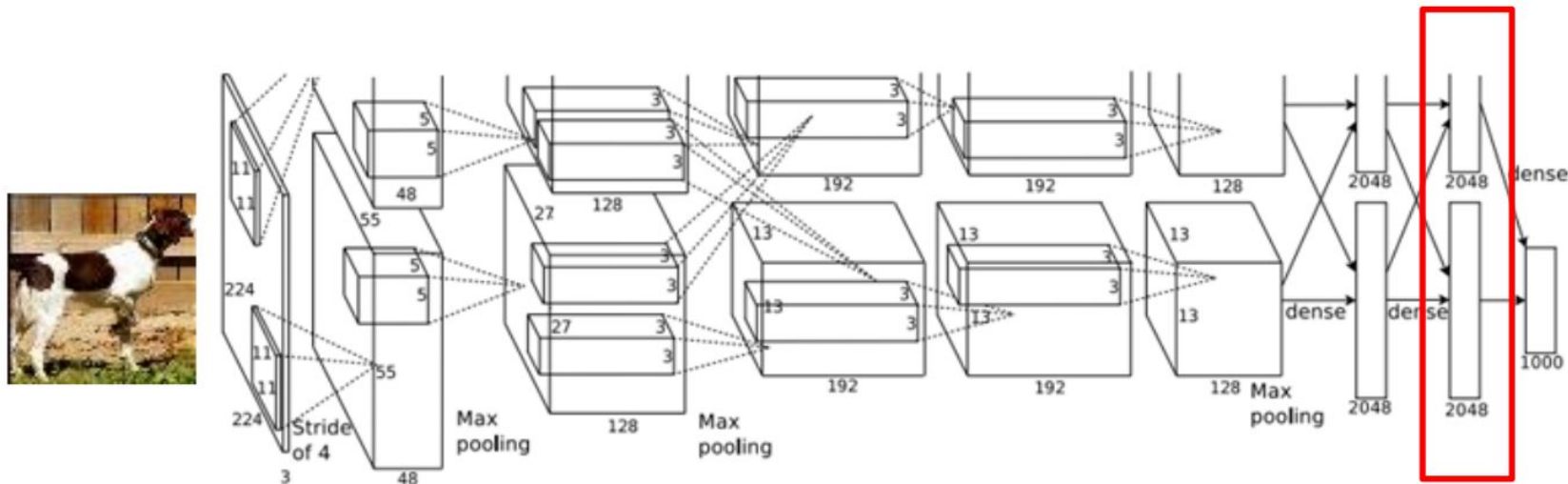
4096-dim vector



Trained model embeddings

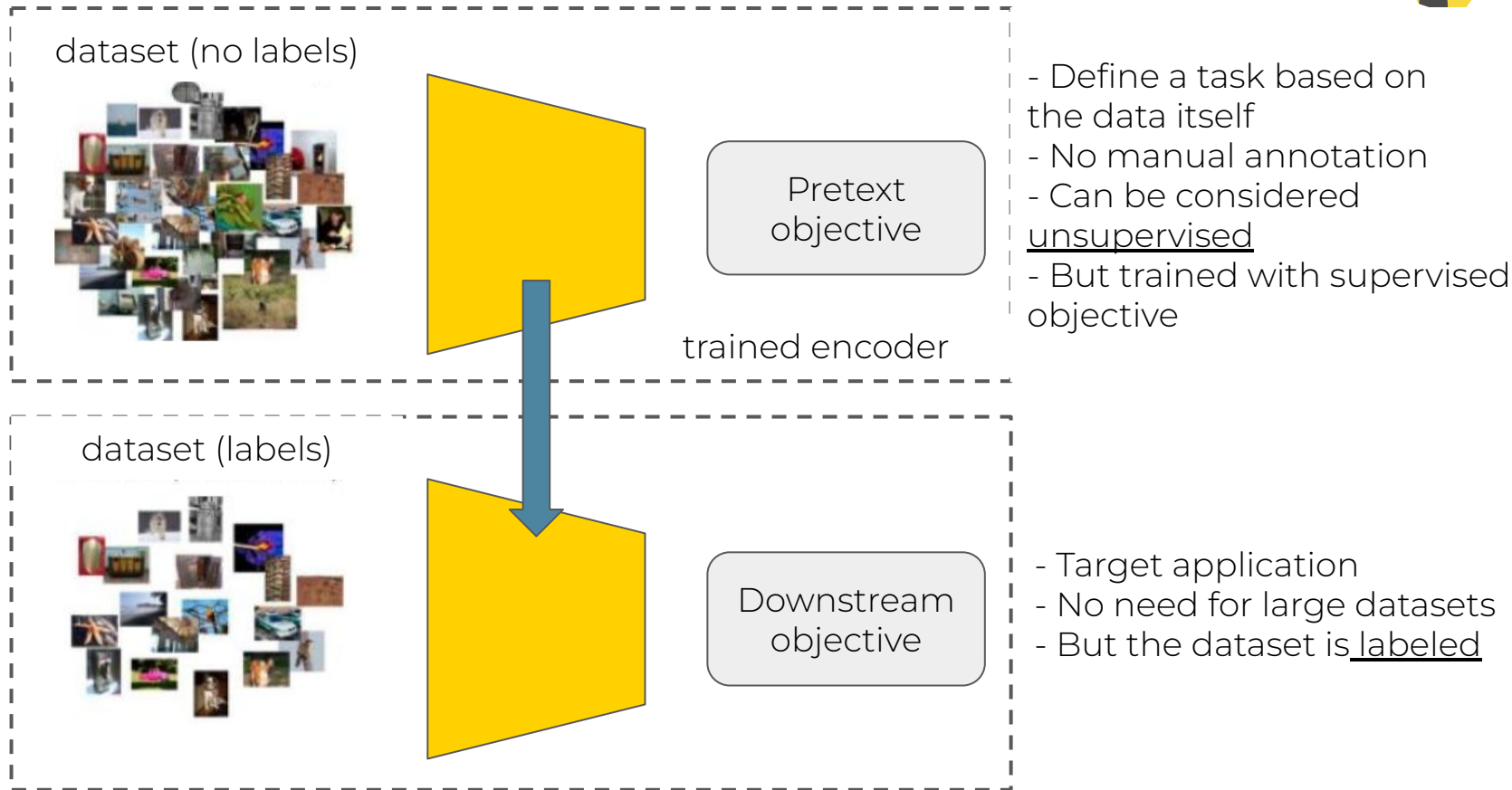


4096-dim vector



Large-scale training: needs a lot of labeled data
Can we do something without huge manually labeled datasets?

Self-supervised learning



Self-supervised: Pretext task



dataset (no labels)



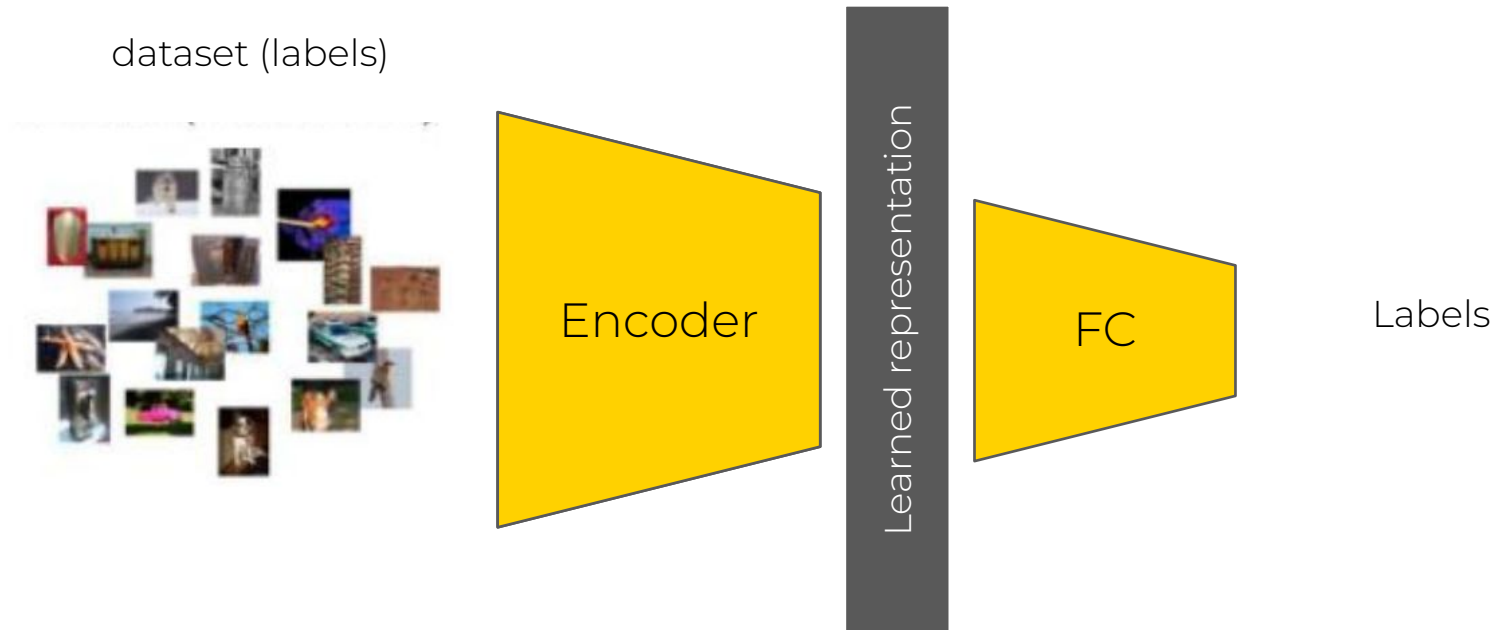
Encoder

Learned representation

Decoder,
Classifier,
Regressor

Labels/outputs
automatically
generated
from data

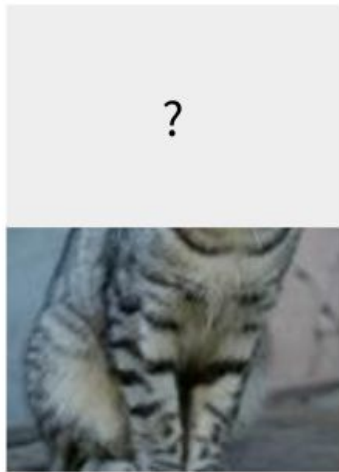
Self-supervised: Downstream task



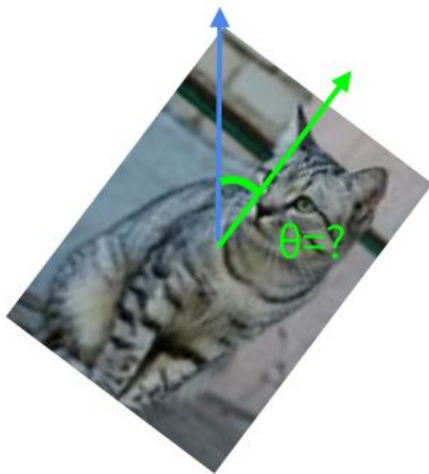
Self-supervised: Pretext tasks



Example: learn to predict image transformations / complete corrupted images



completion



rotation
prediction



“jigsaw” puzzle



colorisation



Self-supervised: Evaluation

- Pretext Task Performance
 - Measure how well the model performs on the task it was trained on without labels.
- Representation Quality
 - Evaluate the quality of the learned representations
 - Linear Evaluation Protocol: Train a linear classifier on the learned representations
 - Clustering: Measure clustering performance;
- Transfer Learning and Downstream Task Performance
 - Assess the utility of the learned representations by transferring them to a downstream supervised task.

Self-supervised: Evaluation

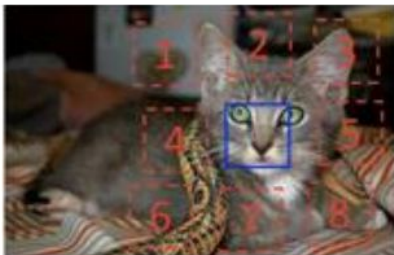


- Robustness and Generalization
 - Test how well the model generalizes to different datasets and is robust to variations.
- Computational Efficiency
 - Assess the efficiency of the method in terms of training time and resource requirements.

Broader picture



computer vision



Doersch et al. 2015

speech synthesis



Wavenet (van den Oord
et al., 2016)

language modeling

GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

GPT-4 (OpenAI 2023)

...

Pretext example: predict rotations



90° rotation



270° rotation



180° rotation



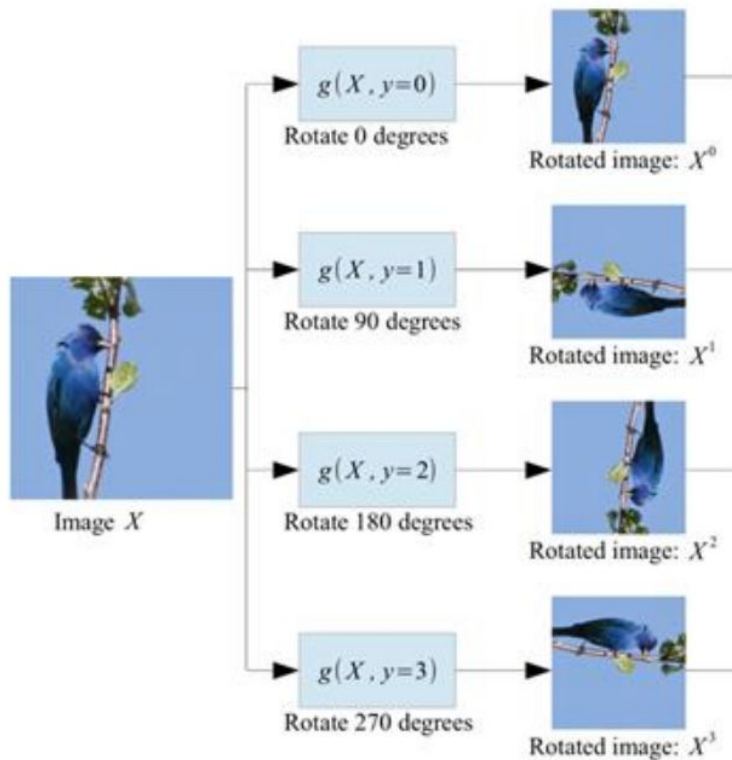
0° rotation



270° rotation

Hypothesis: a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

Pretext example: predict rotations

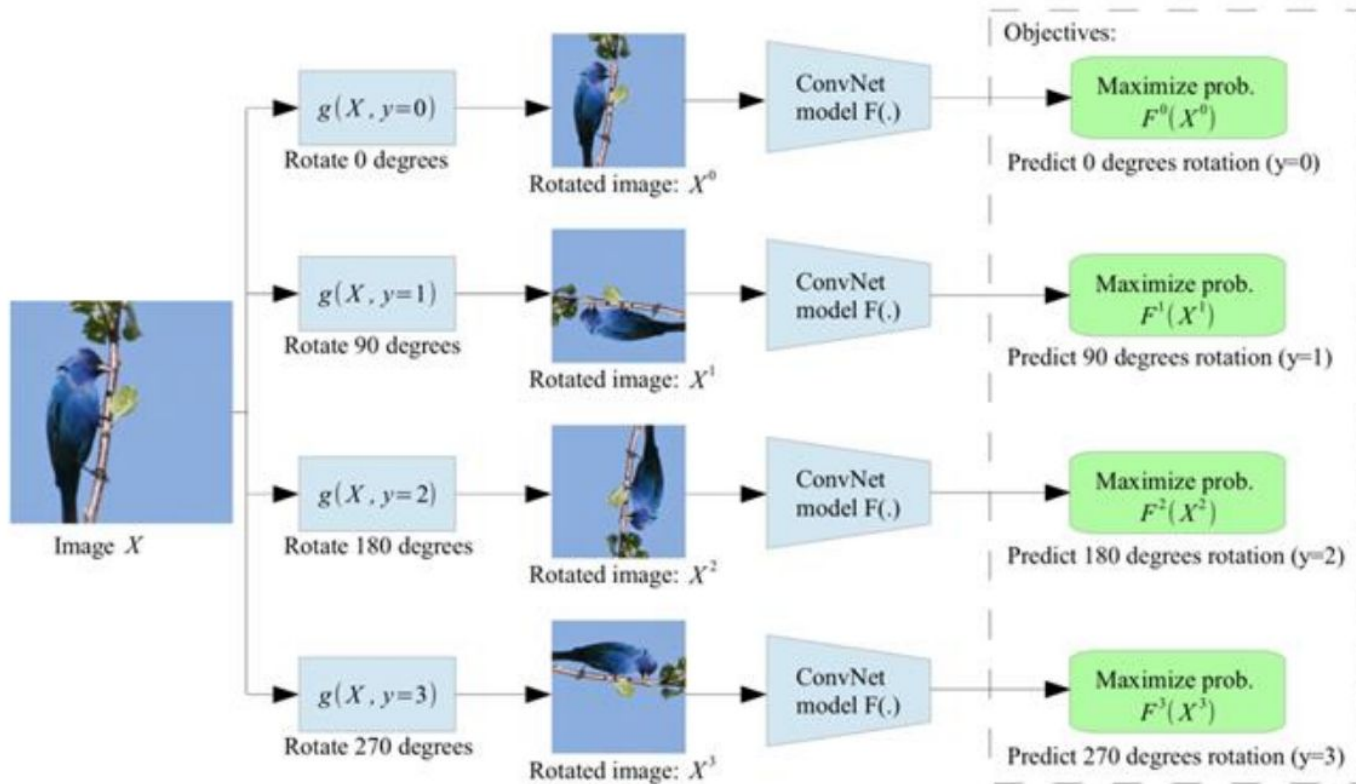


The model learns to predict which rotation is applied (4-class classification)

(Image source: [Cidaris et al. 2018](#))

[slide credit](#)

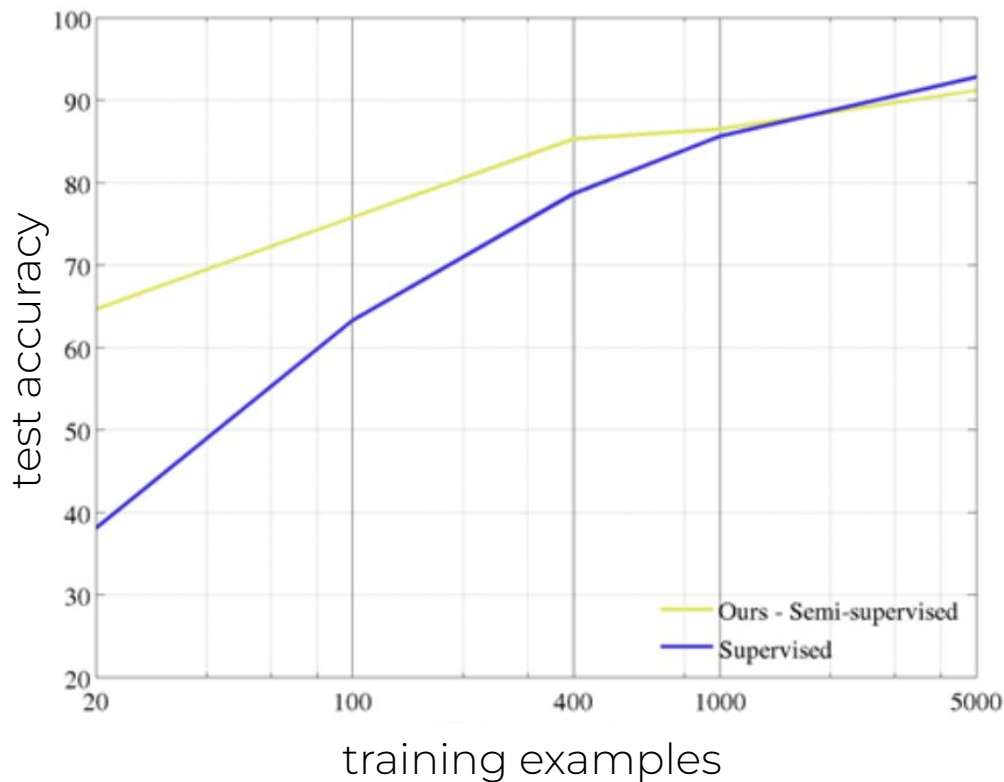
Pretext example: predict rotations



(Image source: [Cidaris et al. 2018](#))

[slide credit](#)

Evaluation



Self-supervised learning on
CIFAR10 (entire training set).

Freeze conv1 + conv2

Learn conv3 + linear layers
with subset of labeled
CIFAR10

(Image source: [Cidaris et al. 2018](#)) [slide credit](#)

Evaluation



	Classification (%mAP)		Detection (%mAP)	Segmentation (%mIoU)
Trained layers	fc6-8	all	all	all
ImageNet labels	78.9	79.9	56.8	48.0
Random		53.3	43.4	19.8
Random rescaled Krähenbühl et al. (2015)	39.2	56.6	45.6	32.6
Egomotion (Agrawal et al., 2015)	31.0	54.2	43.9	
Context Encoders (Pathak et al., 2016b)	34.6	56.5	44.5	29.7
Tracking (Wang & Gupta, 2015)	55.6	63.1	47.4	
Context (Doersch et al., 2015)	55.1	65.3	51.1	
Colorization (Zhang et al., 2016a)	61.5	65.6	46.9	35.6
BIGAN (Donahue et al., 2016)	52.3	60.1	46.9	34.9
Jigsaw Puzzles (Noroozi & Favaro, 2016)	-	67.6	53.2	37.6
NAT (Bojanowski & Joulin, 2017)	56.7	65.3	49.4	
Split-Brain (Zhang et al., 2016b)	63.0	67.1	46.7	36.0
ColorProxy (Larsson et al., 2017)		65.9		38.4
Counting (Noroozi et al., 2017)	-	67.7	51.4	36.6
(Ours) RotNet	70.87	72.97	54.4	39.1

Pretrained with full ImageNet supervision

No pretraining

Self-learning on
ImageNet (training set)
with AlexNet

Finetune on labeled data
from Pascal VOC 2007.

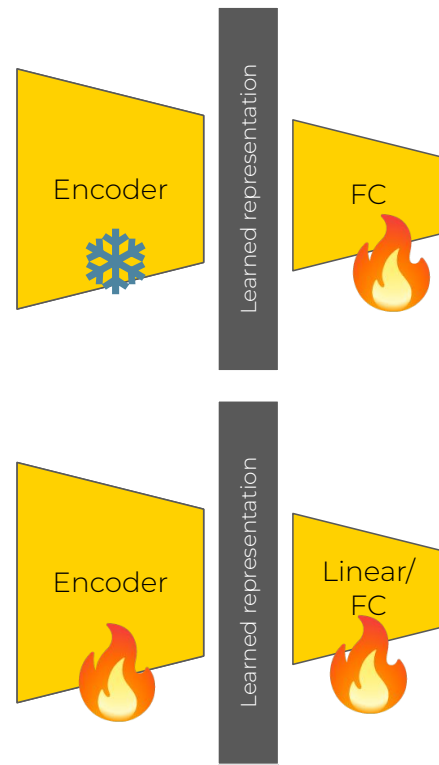
Downstream task: training

Linear probing:

- pre-trained model is fixed
- only one linear layer is added on top to produce the output
- assess the quality of representations from a pre-trained feature extraction model

Full fine-tuning

- pre-trained model is further trained (not fixed).
- one or more layers, possibly with non-linearities are added.



Pretext tasks



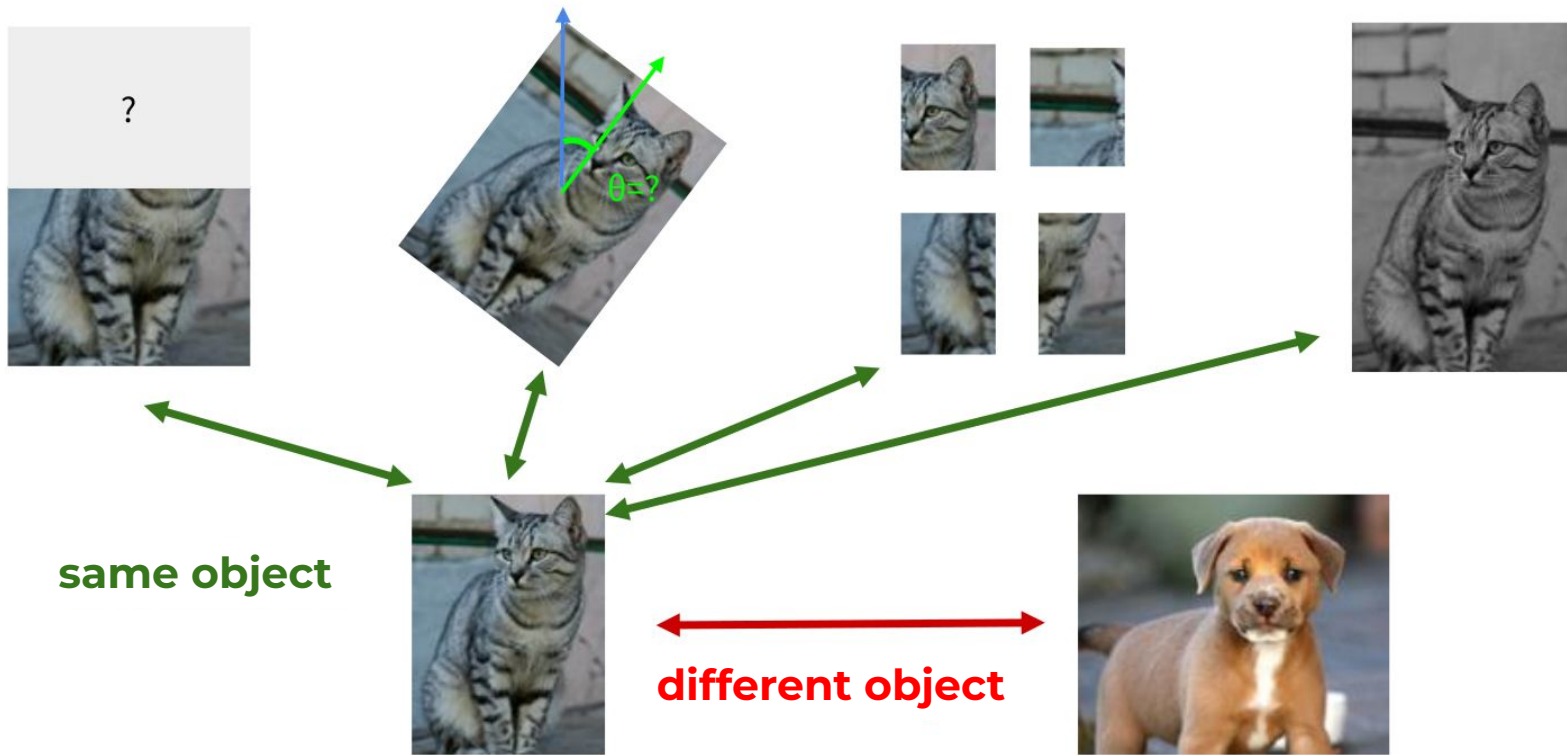
Summary:

- The models are forced learn good features about natural images, e.g. semantic representations, in order to solve the pretext tasks
- We often do not care about the performance of these pretext tasks, but rather about the learned features for downstream tasks

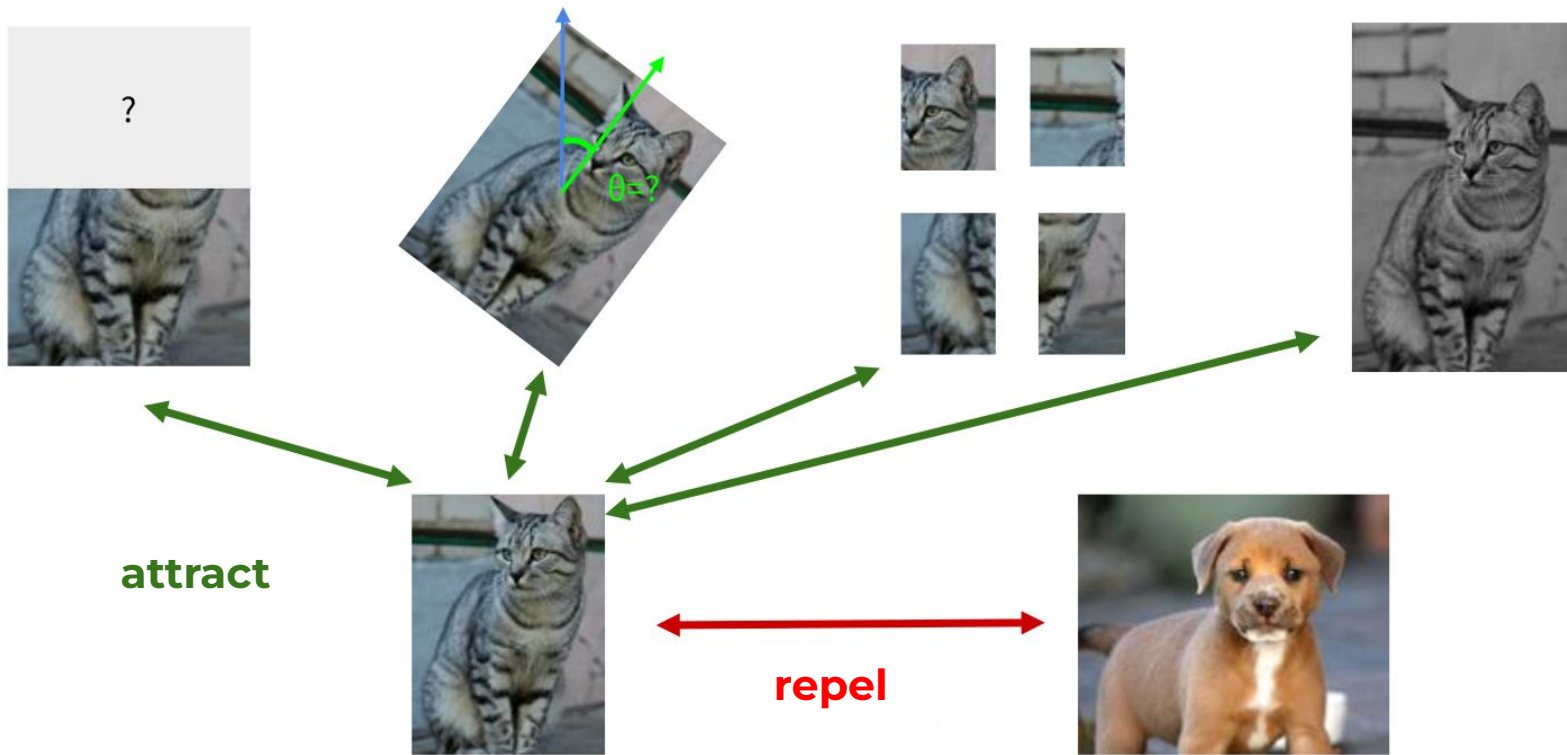
Problems:

- coming up with individual pretext tasks is tedious
- the learned representations may not be general

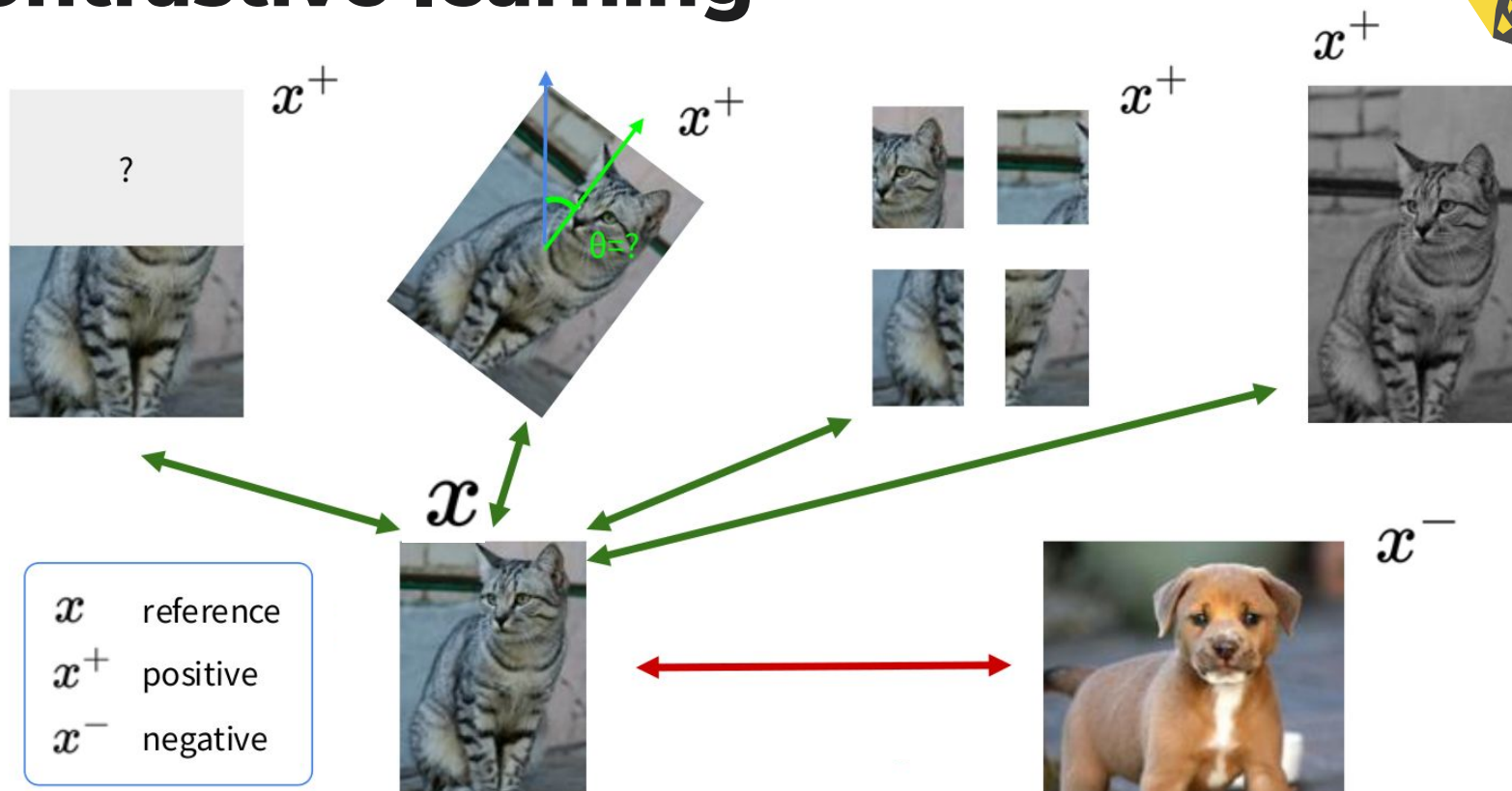
More general pretext task



Contrastive learning



Contrastive learning





Contrastive learning: formulation

What we want

$$\textit{score}(f(x), f(x^+)) \gg \textit{score}(f(x), f(x^-))$$

- x : reference sample
- x^+ positive sample
- x^- negative sample

Given a chosen score function, we aim to learn an encoder function $f()$ that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) .

Contrastive learning: formulation



Loss function given 1 positive and N-1 negative samples

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$



x



x^+



x



x_1^-



x_2^-



x_3^-

...

Contrastive learning: formulation



Loss function given 1 positive and N-1 negative samples

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{positive pair score}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{positive pair score}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

This seems familiar ...

Cross entropy loss for a N-way softmax classifier!

i.e., learn to find the positive sample from the N samples

Contrastive learning: formulation



Loss function given 1 positive and N-1 negative samples

$$L = -\mathbb{E}_X \left[\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{positive pair score}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{positive pair score}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for the N-1 negative pairs}}} \right]$$

Commonly known as the InfoNCE loss ([van den Oord et al., 2018](#))

A lower bound on the mutual information between $f(x)$ and $f(x^+)$

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

The larger the negative sample size (N), the tighter the bound



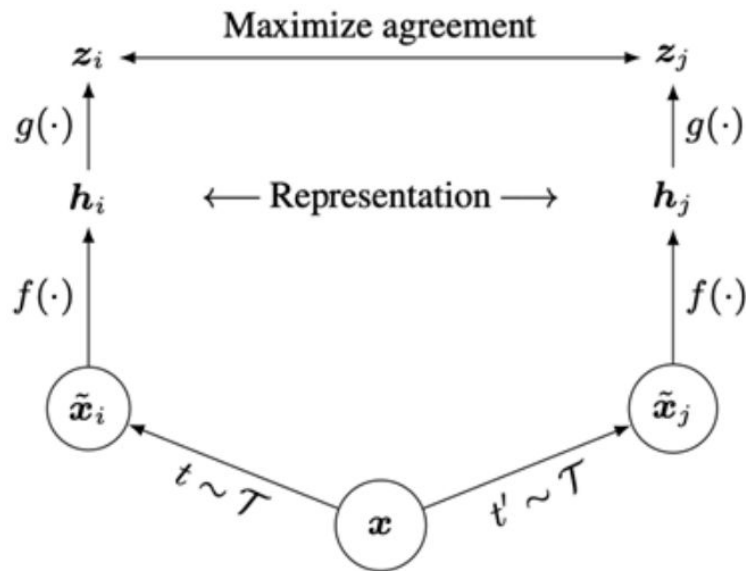
SimCLR

Cosine similarity as the score function:

$$s(u, v) = \frac{u^T v}{||u|| ||v||}$$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied

Generate positive samples through data augmentation: random cropping, random color distortion, and random blur.



SimCLR: positives by augmentation



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise



(i) Gaussian blur

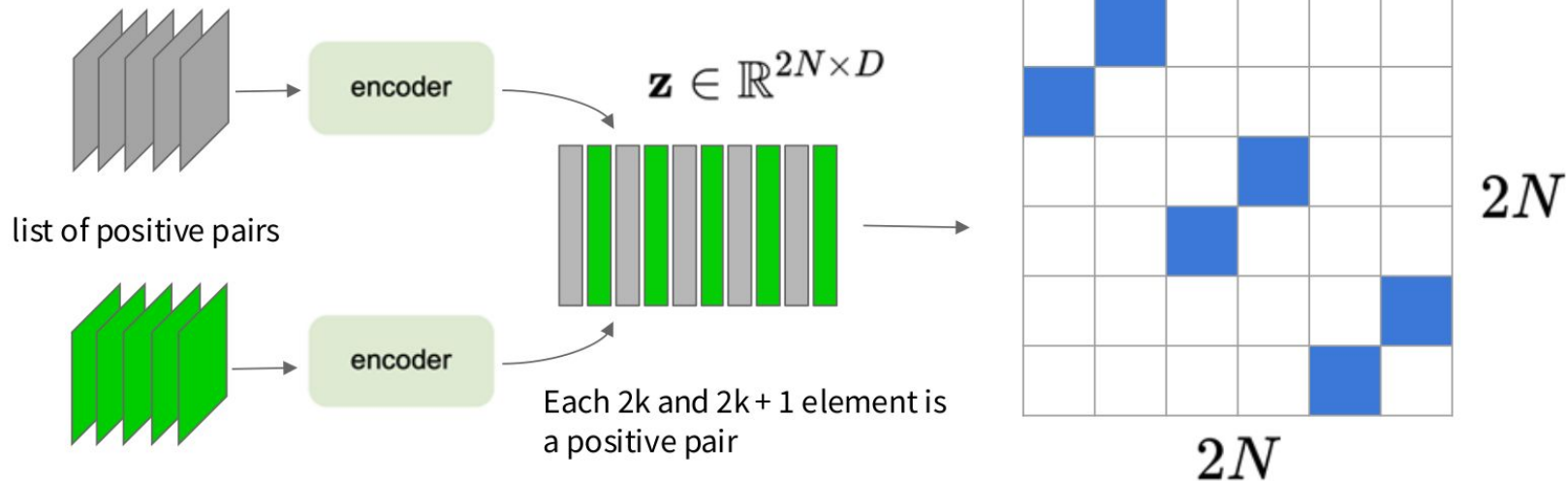


(j) Sobel filtering



SimCLR

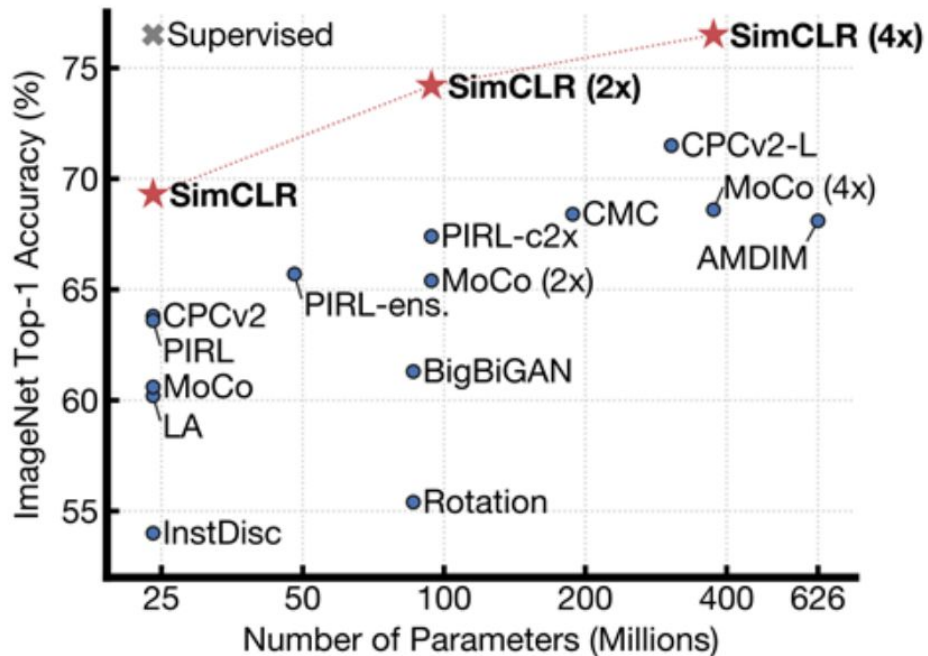
SimCLR: mini-batch training



 = classification label for each row



SimCLR: training on features



Train feature encoder on ImageNet (entire training set) using SimCLR.

Freeze feature encoder, train a linear classifier on top with labeled data.



SimCLR: semi-supervised

Method	Architecture	Label fraction	
		1%	10%
		Top 5	
Supervised baseline	ResNet-50	48.4	80.4
<i>Methods using other label-propagation:</i>			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
<i>Methods using representation learning only:</i>			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	85.8	92.6

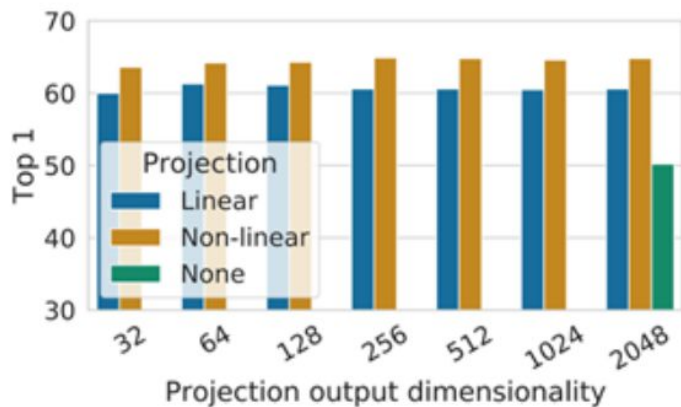
Table 7. ImageNet accuracy of models trained with few labels.

Train feature encoder on ImageNet (entire training set) using SimCLR.

Finetune the encoder with 1% / 10% of labeled data on ImageNet.



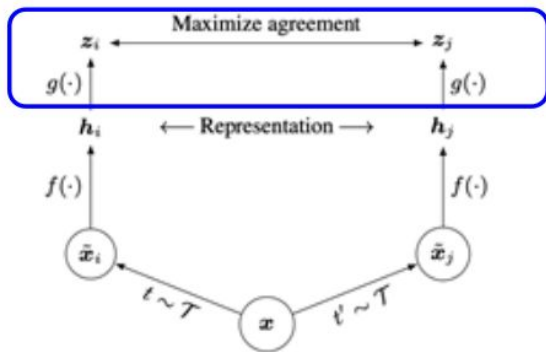
SimCLR: projection head



Linear / non-linear projection heads improve representation learning!

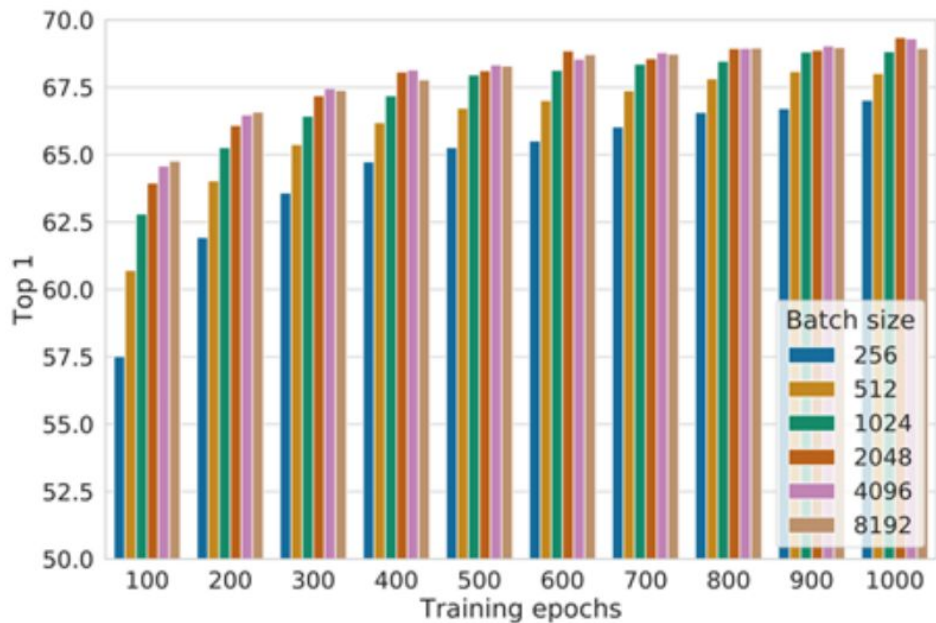
A *possible* explanation:

- contrastive learning objective may discard useful information for downstream tasks
- representation space z is trained to be invariant to data transformation.
- by leveraging the projection head $g(\cdot)$, more information can be preserved in the h -representation space





SimCLR: large batch size



Large training batch size is crucial for SimCLR!

Large batch size causes large memory footprint during backpropagation: requires distributed training on TPUs (ImageNet experiments)

Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

Summary



- Word vectors are simply vectors of numbers that represent the meaning of a word
- Image vectors also represent “meanings” and reused for downstream tasks
- One of the goals of modern deep learning is to find ways to generate meaningful vectors for all kinds of input data

Revise



1. Word embeddings
2. Word2vec
3. Self-supervised learning
4. Contrastive learning

Thanks for attention!

Questions?



girafe
ai

